

ALGORITMI PARALELI ȘI DISTRIBUIȚI

Curs

Facultatea de Electronică, Comunicații și Calculatoare
Universitatea Națională de Știință și Tehnologie Politehnica
București – Centrul universitar Pitești

Programare paralelă și distribuită cu MPI

1. Comunicarea prin mesaje

2. MPI: Concepte de bază și definiții

3. Fundamente MPI

- Structura unui program MPI
- Comunicatori și grupuri de procese
- Moduri de comunicație în MPI

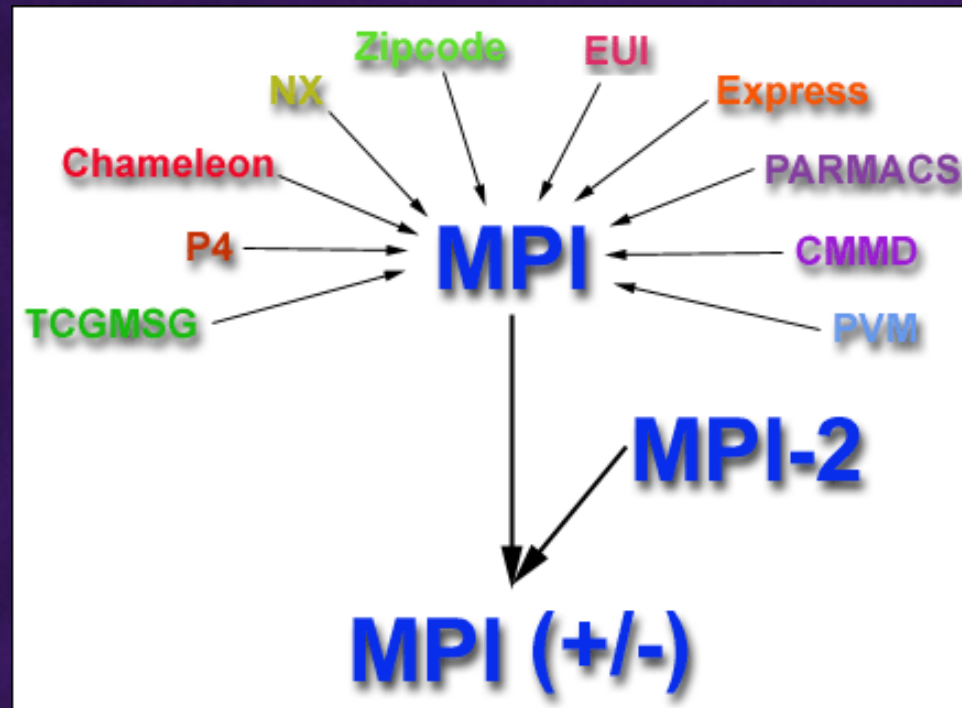
4. Exemple

Comunicarea prin mesaje

Comunicarea prin mesaje

1. Programul este divizat într-o mulțime de **procese concurente**.
2. Procesele comunica între ele **transmiterea și recepția mesajelor**.
3. Transferul de informații are loc prin execuția operațiilor de tip cooperativ.
4. Transfer de mesaje = **programare explicită**.
5. **Limbaje specializate:** ADA, OCCAM, LINDA, ORCA, PARLOG, etc
6. **Biblioteci specializate:** MPI, PVM, etc.

MPI – Concepte de bază și definiții



MPI ≠ LIMBAJ DE PROGRAMARE

MPI – Concepte de bază și definiții

Motive utilizare MPI

1. **STANDARDIZARE.**
2. **PORTABILITATE**
3. **PERFORMANȚĂ**
4. **FUNCȚIONALITATE**
5. **DISPONIBILITATE**

Fundamente MPI

- Structura unui program MPI
- Comunicatori și grupuri de procese
- Moduri de comunicație în MPI

Structura unui program MPI

Header MPI

Variabile MPI

Inițializare MPI

Calcul și Comunicare
MPI

Finalizare program MPI

```
#include <mpi.h>
```

```
void main(int argc, char* argv[]) {  
    int ProcNum = 0; // Nr. de procese  
    int ProcRank = 0; // Rangul fiecarui proces  
    int eroare;
```

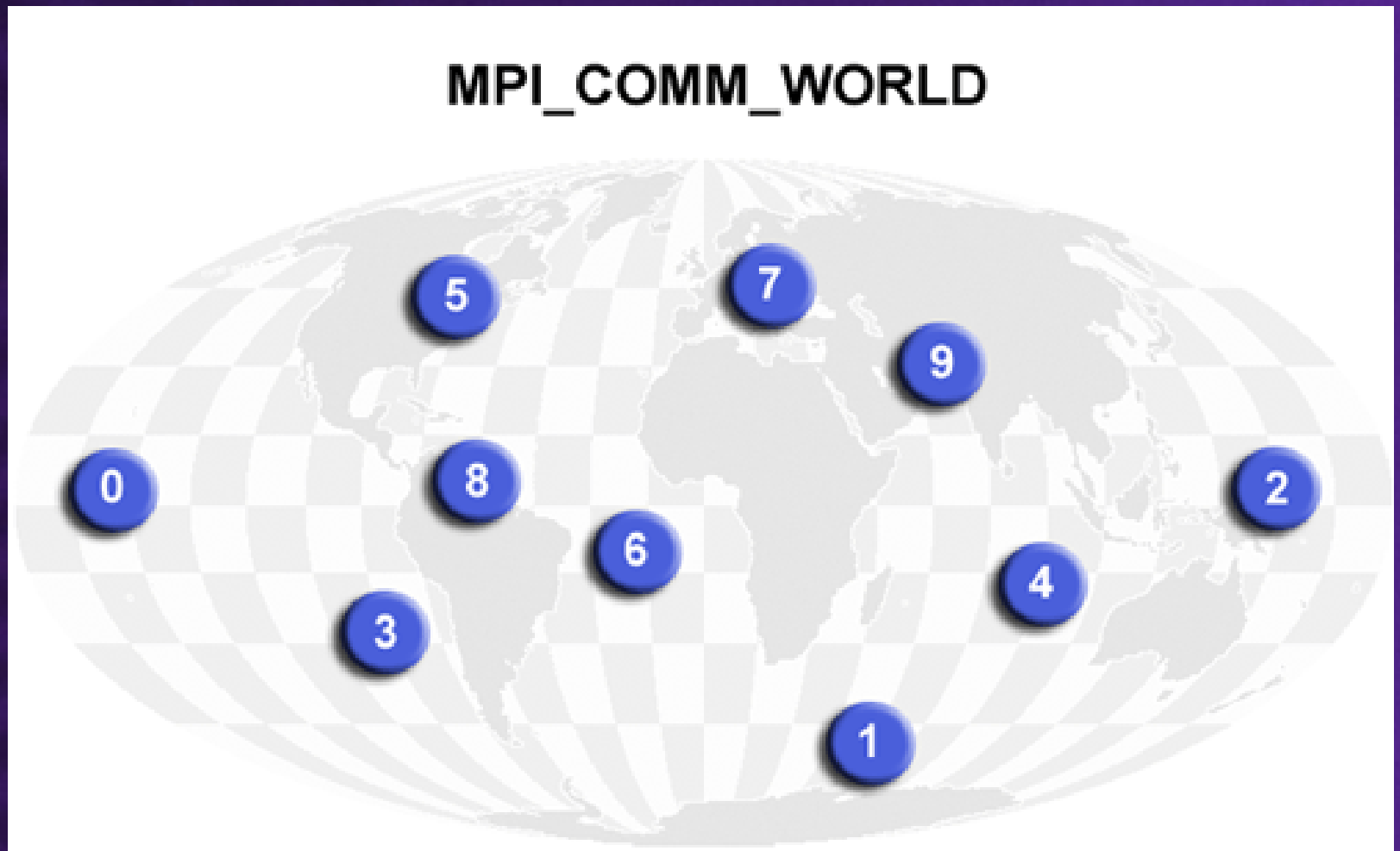
```
    eroare = MPI_Init(&argc, &argv);
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &ProcNum);  
    MPI_Comm_rank(MPI_COMM_WORLD, &ProcRank);
```

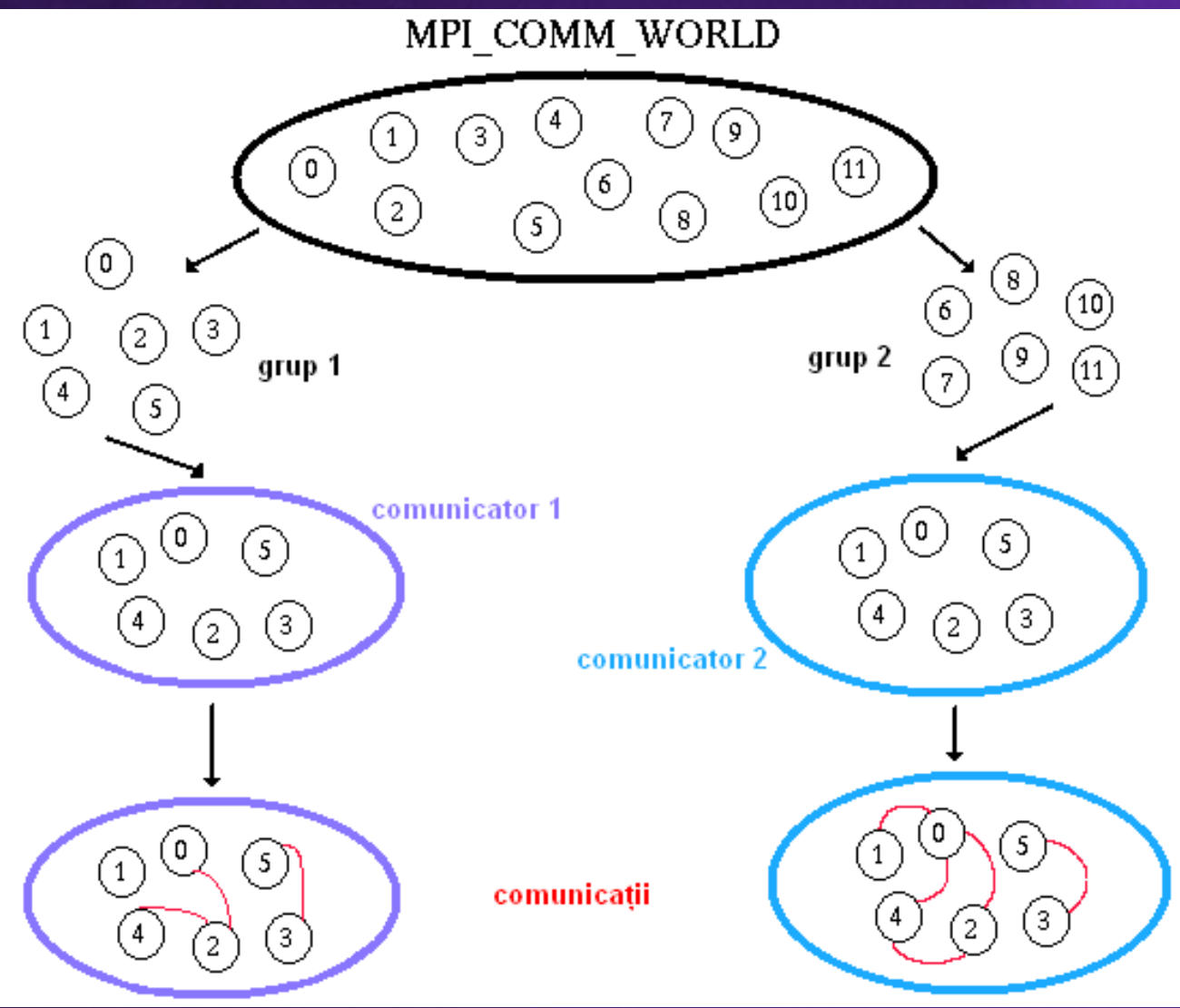
```
    // Calcule & Mesaje  
    <...>
```

```
    MPI_Finalize();  
}
```


Comunicatori și grupuri de procese



Comunicatori și grupuri de procese



Comunicatori și grupuri de procese

Exemplu

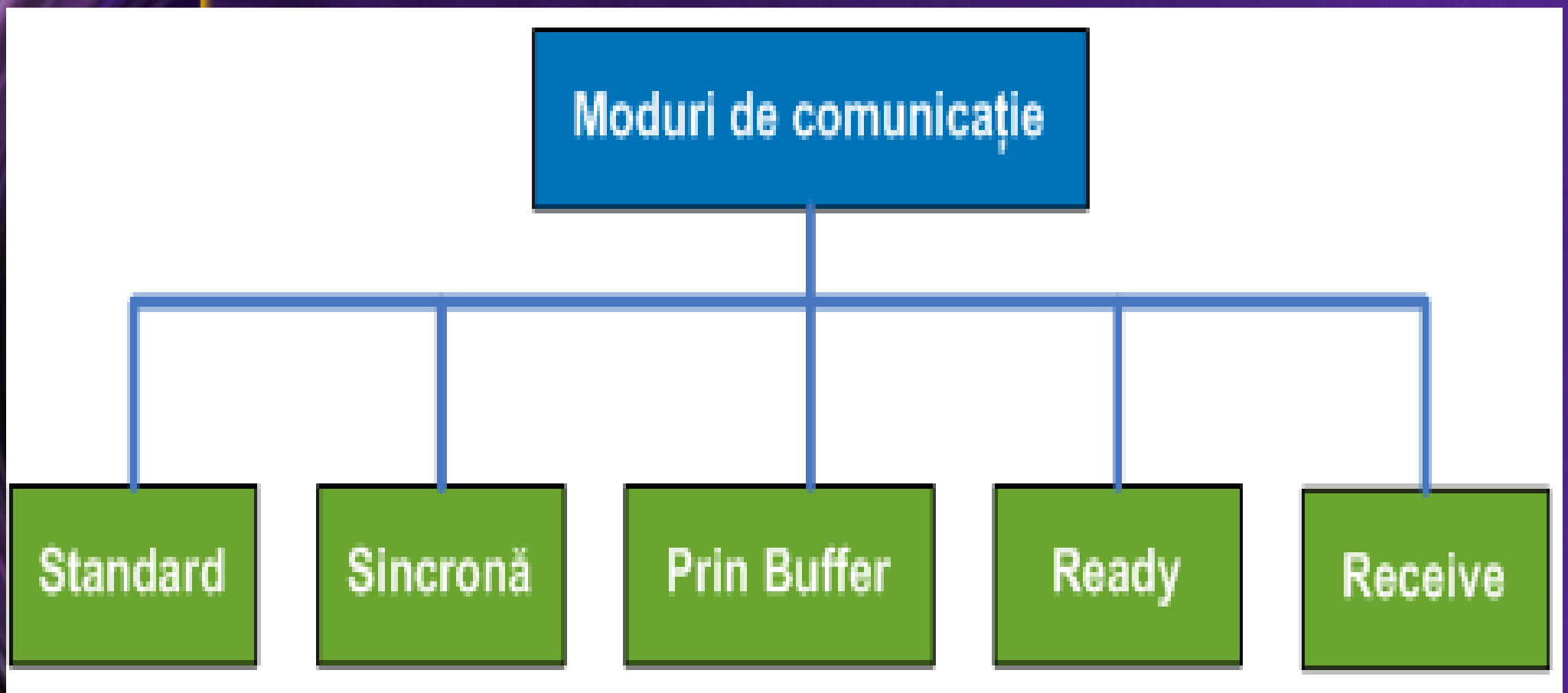
```
#include "mpi.h" #include <stdio.h>

void main(int argc, char *argv[])
{
    int nrtasks, rank, rc;

    rc = MPI_Init(&argc, &argv);
    if (rc != MPI_SUCCESS) {
        printf ("Eroare start program MPI.\n");
        MPI_Abort(MPI_COMM_WORLD, rc);
    }

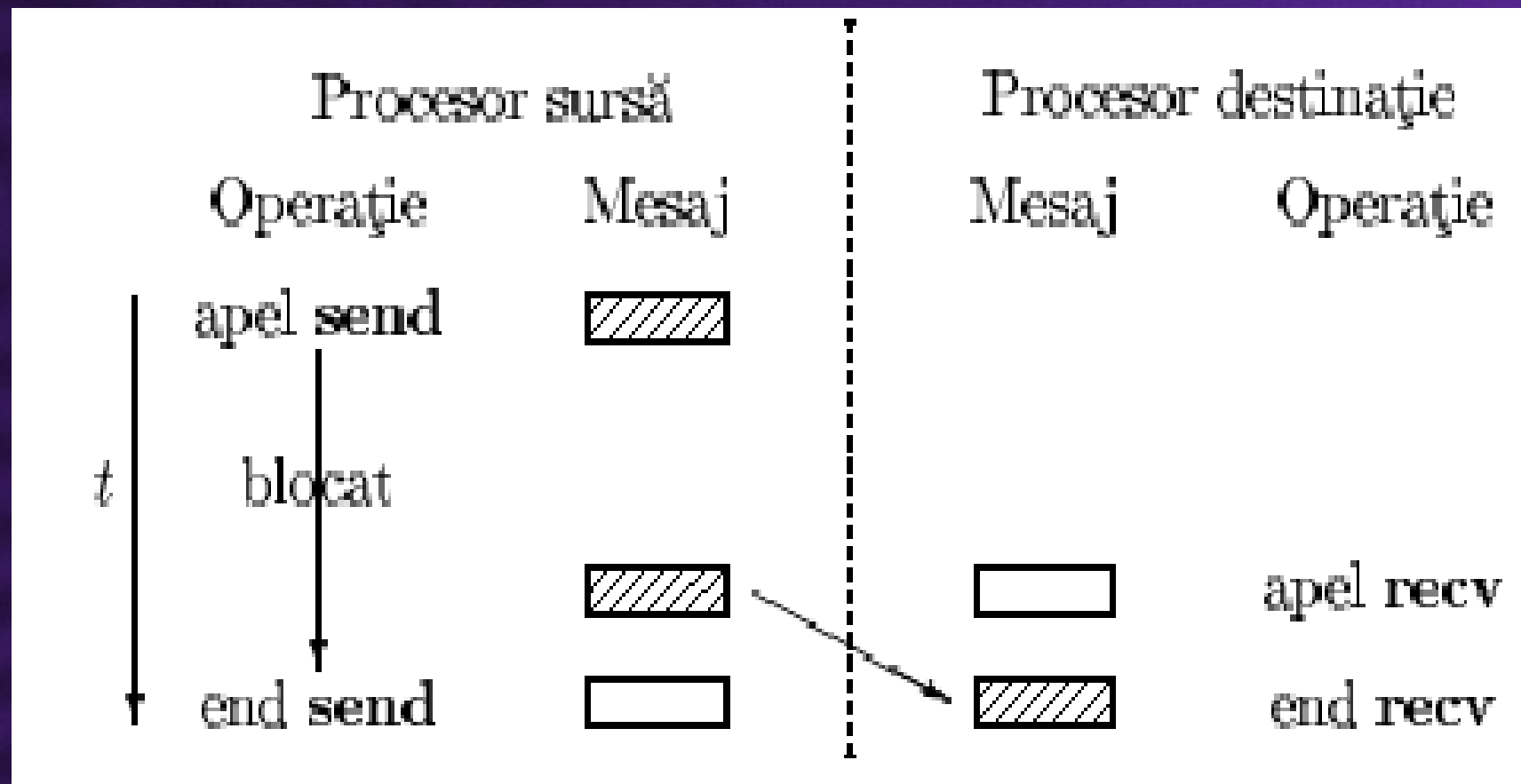
    MPI_Comm_size(MPI_COMM_WORLD, &nrtasks);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    printf ("Nr. de task-uri= %d My rank= %d\n",
            nrtasks, rank);
    MPI_Finalize();
}
```

Moduri de comunicație în MPI

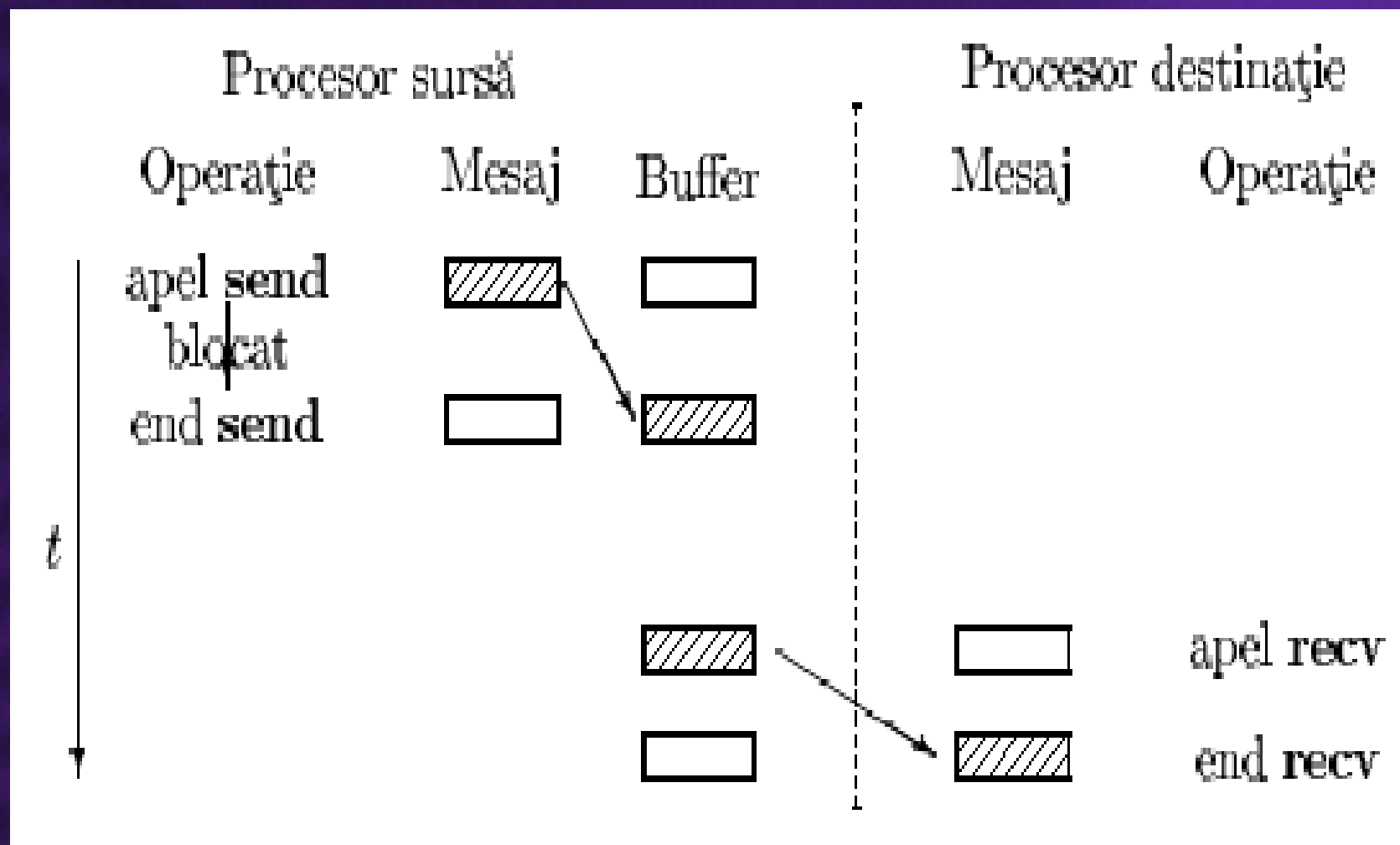


Moduri de comunicație în MPI

1. COMUNICAȚIE SINCRONĂ (BLOCANTĂ).



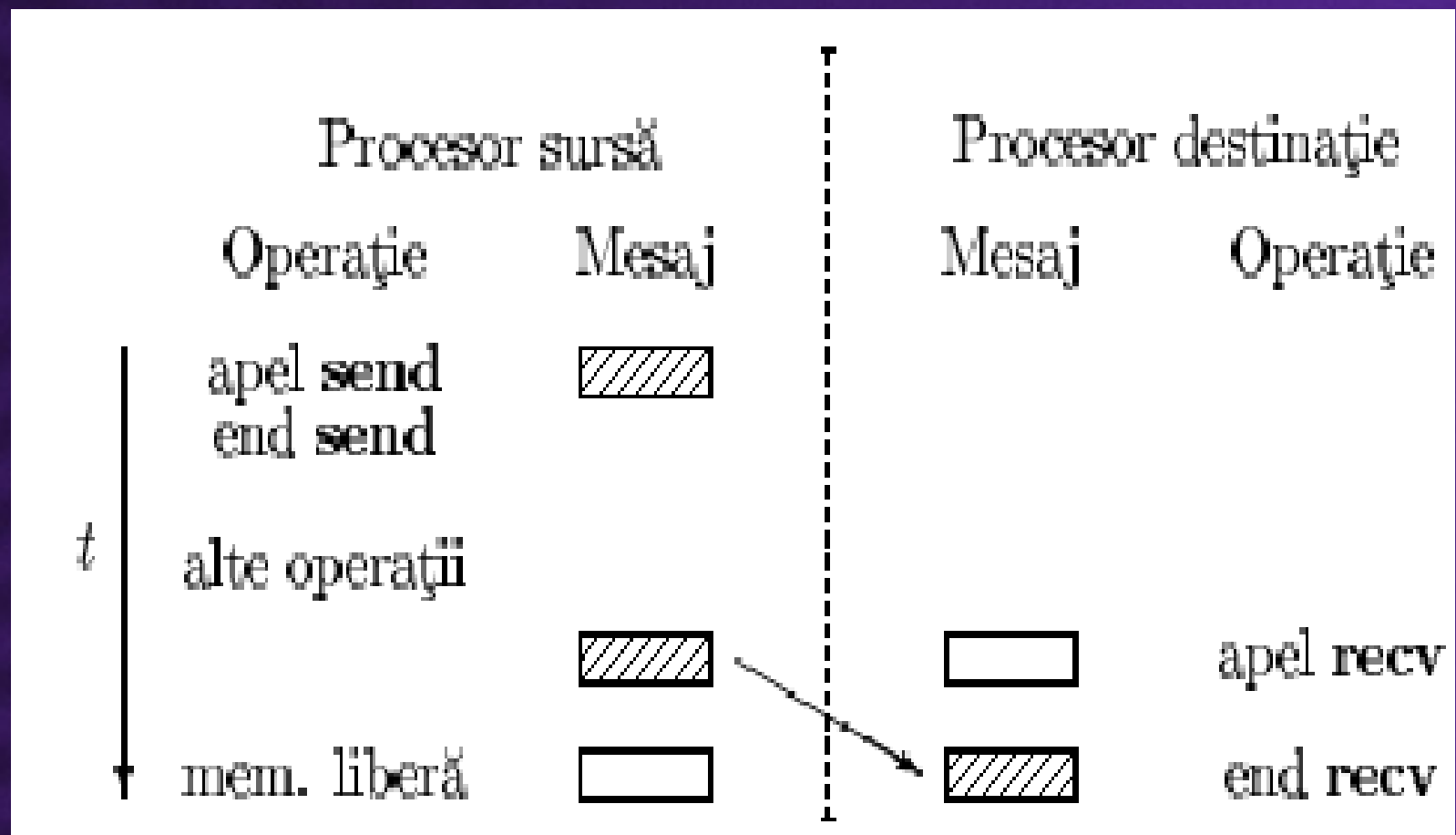
Moduri de comunicație în MPI



Comunicație blocantă prin buffer

Moduri de comunicație în MPI

2. COMUNICAȚIE ASINCRONĂ (NEBLOCANTĂ)



Comunicație neblocantă prin buffer

Moduri de comunicație în MPI

Moduri de comunicație

Comunicație	Blocantă	Neblocantă
Standard	MPI_Send MPI_Recv	MPI_Isend MPI_Irecv
Sincronă	MPI_Ssend	MPI_Issend
Prin buffer	MPI_Bsend	MPI_Ibsend
Ready	MPI_Rsend	MPI_Irsend

Moduri de comunicație în MPI

Transmisie blocantă standard **MPI_Send**.

```
MPI_Send (  
    void *buffer, int count,  
    MPI_Datatype datatype, int dest,  
    int tag, MPI_Comm comm)
```

Recepție blocantă standard **MPI_Recv**.

```
int MPI_Recv (  
    void *buffer, int count,  
    MPI_Datatype datatype, int source,  
    int tag, MPI_Comm comm,  
    MPI_Status status)
```

Moduri de comunicație în MPI

Transmisie blocantă sincronă **MPI_Ssend**.

```
int MPI_Ssend (void * buffer, int count, MPI_Datatype datatype, int  
               dest, int tag, MPI_Comm comm)
```

Transmisie blocantă ready **MPI_Rsend**.

```
int MPI_Rsend (void *buffer, int count, MPI_Datatype datatype, int  
               dest, int tag, MPI_Comm comm)
```

Transmisie blocantă prin buffer **MPI_Bsend**.

```
int MPI_Bsend (void *buffer, int count, MPI_Datatype datatype,  
               int dest, int tag, MPI_Comm comm)
```

```
int MPI_BUFFER_ATTACH (void *buffer, int size).
```

```
int MPI_BUFFER_DETACH (void *buffer, int size).
```

Moduri de comunicație în MPI

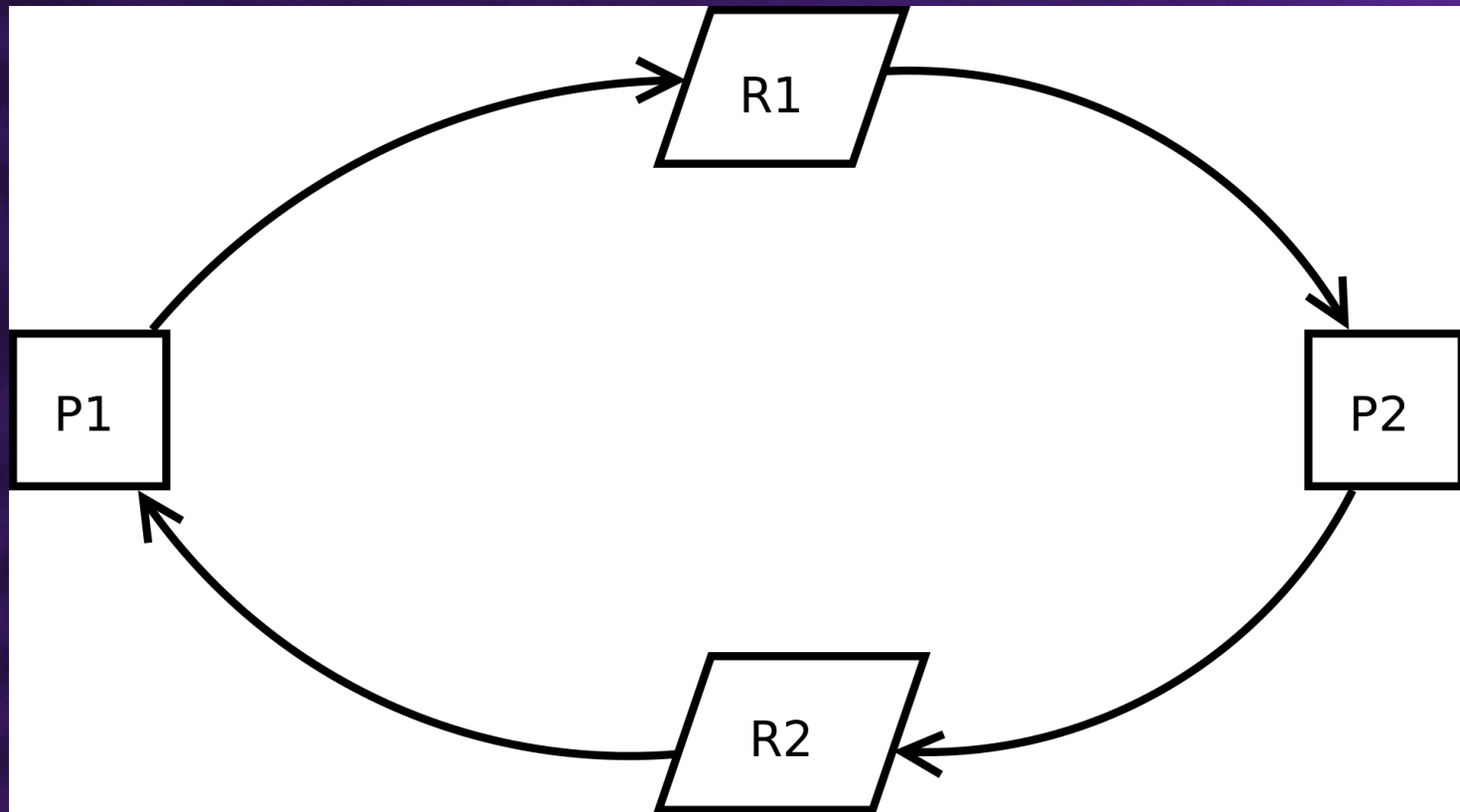
Transmisie **neblokantă standard** **MPI_Isend**.

```
int MPI_Isend (  
    void *buffer, int count,  
    MPI_Datatype datatype, int dest,  
    int tag, MPI_Comm comm,  
    MPI_Request *req);
```

Recepție **neblokantă standard** **MPI_Irecv**.

```
int MPI_Irecv (  
    void *buffer, int count,  
    MPI_Data type datatype, int source,  
    int tag, MPI_Comm comm,  
    MPI_Request *req);
```


Blocaje MPI

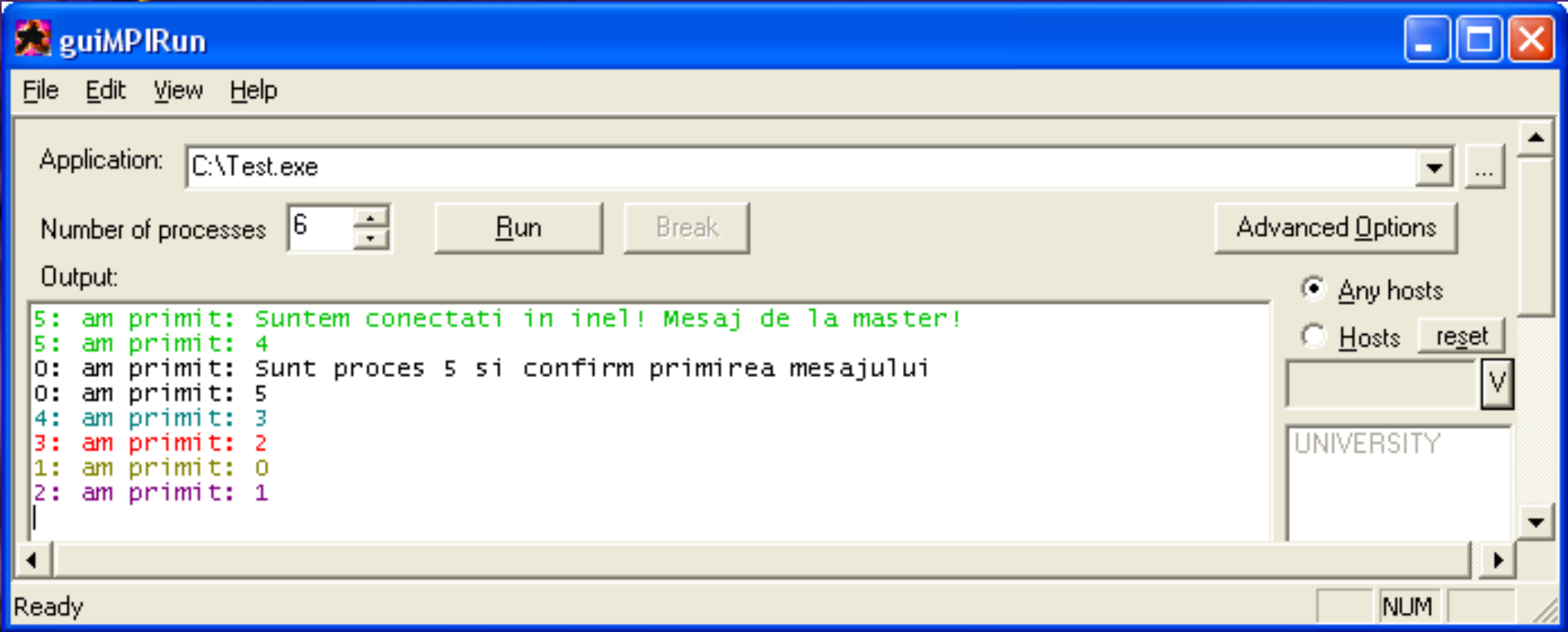


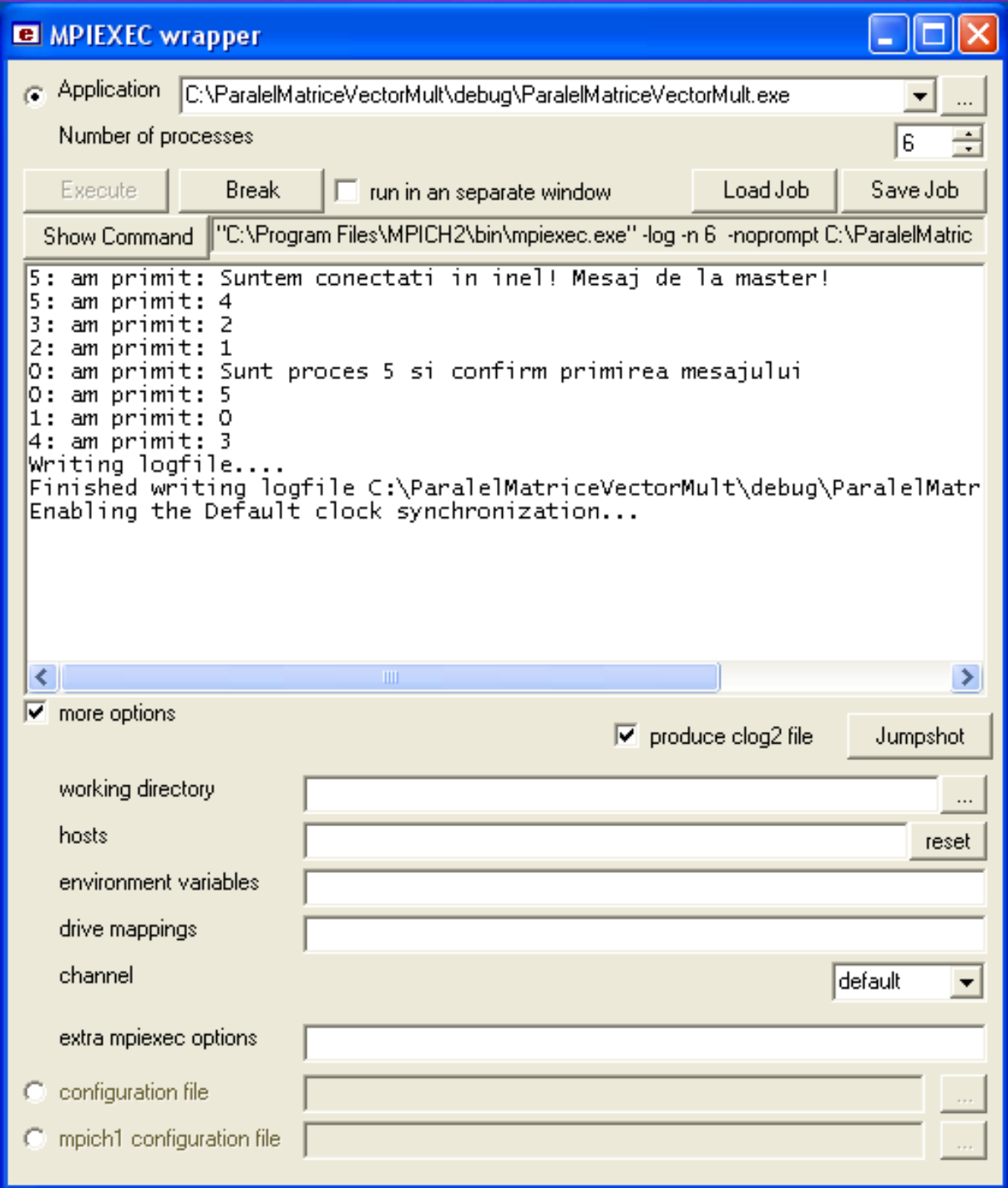
<https://en.wikipedia.org/wiki/Deadlock>

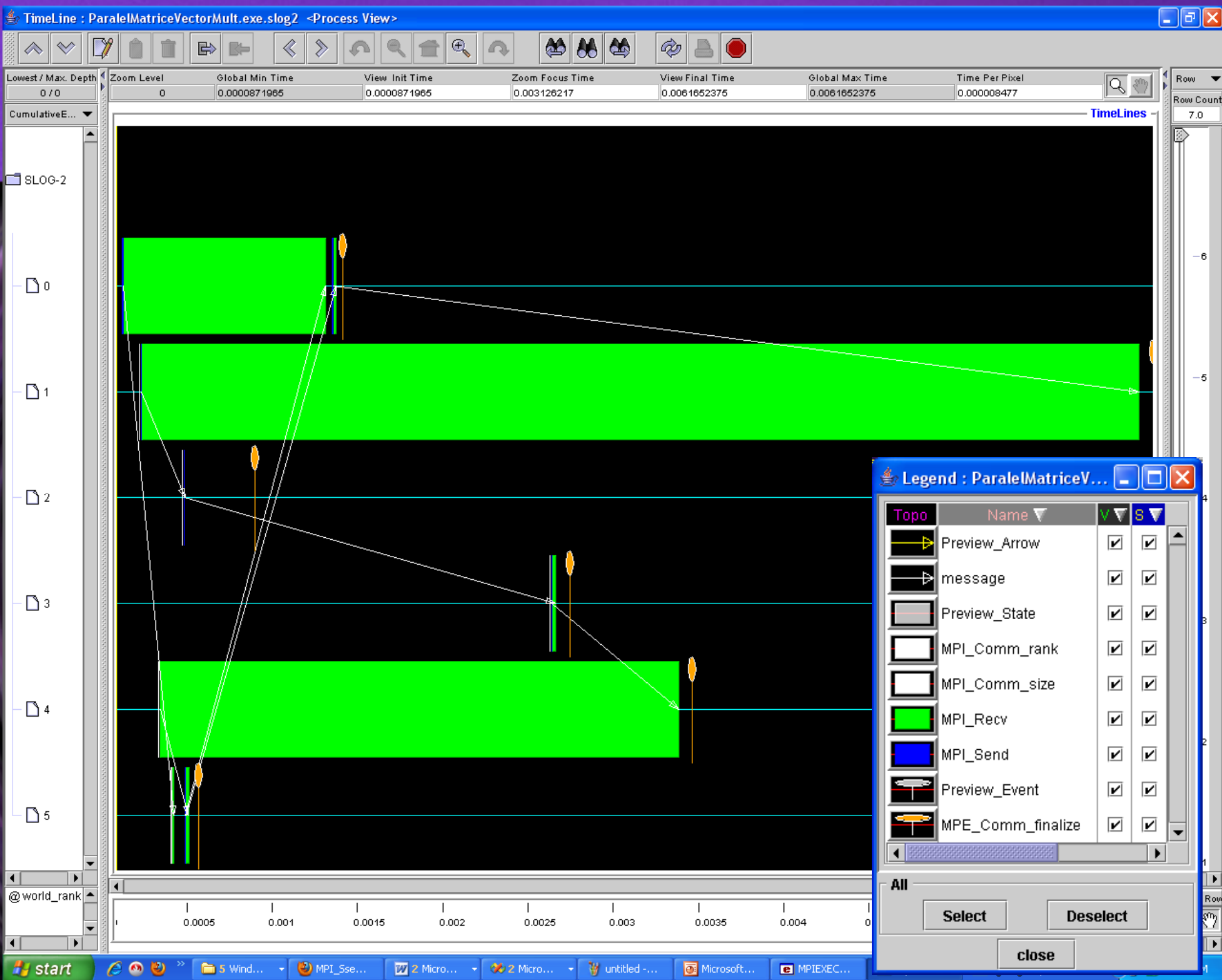
Aplicație

Program MPI ce implementează operația de transmitere a unui mesaj de către fiecare proces vecinului sau din dreapta (se considera o topologie de tip inel).

Mesajul consta in id-ul fiecarui proces. Inainte de trimiterea id-urilor, se va realiza un schimb de mesaje intre procesul master (0) si procesul ce inchide inelul (p-1).







Aplicație

Exemplificare procedura rezolvare
pe tabla

Program MPI ce implementeaza operatia de transmitere a unor mesaje intre procesele/procesoarele ce compun sistemul cluster.

1. Trimitere mesaj de la master la toate celelalte procese/procesoare (ex. Mesaj: "Va salut! Sa incepem lucrul!") .
2. Trimitere valoare de tip intreg – citita de la tastatura - de la master la procesorul cu ID-ul =2.
3. Procesorul cu ID-ul 2 primeste data trimisa de master si adauga la ce a primit valoarea ID-ului sau.
4. Procesorul cu ID-ul 2 trimite inapoi master-ului noua valoare obtinuta.
5. Master afiseaza valoarea primita de la proc. cu ID=2.